

ESPECIFICACIÓN

Proceso de Otorgamiento — y cómo encapsular el motor como servicio

v1.2.1

Este documento hace dos cosas que conviene no mezclar:

- **Parte I — el proceso.** Qué es el otorgamiento, cuándo corre, qué decide y quién actúa. Es descripción de negocio y sirve para validar con Riesgo y con Operaciones.
- **Parte II — el servicio.** Cuál es el contrato del motor si se extrae del navegador y se despliega como microservicio, qué entra, qué sale, qué tiene que recalcular el servidor **aunque el cliente ya lo haya calculado**, y qué falta hoy para llegar ahí. Es especificación técnica.

Están juntos a propósito. La Parte II no se puede escribir sin la I —el contrato del servicio son las decisiones del proceso— y la I sin la II se lee como si el motor pudiera vivir donde está.

La premisa que ordena todo el documento: hoy el motor corre **en el navegador**, y en el navegador **no decide nada**. El atacante es el cliente: con la consola abierta se edita `REGLAS_CLIENTE` y se aprueba lo que sea. Todo lo que sigue está escrito para que el motor pueda mudarse al servidor sin reescribirse — y para dejar anotado lo que todavía lo ata.

Parte I · El proceso

1. Qué es el otorgamiento

El otorgamiento es **la decisión de riesgo sobre una operación concreta**: dado un cliente, sus deudores y el paquete de facturas que se quiere comprar, decide si la operación se cursa, si se cursa **con excepciones autorizadas por un apoderado**, o si no se cursa.

No es lo mismo que la línea. La **línea** dice cuánto cupo hay (`asignarLineas` , spec propio); el otorgamiento dice si el riesgo del cliente y de sus deudores permite usarlo. Una operación puede caber perfectamente en la línea y estar bloqueada por otorgamiento, y al revés.

Tampoco es lo mismo que la **verificación de facturas** (spec propio), que pregunta si el deudor va a pagar *estas* facturas. Las tres rutinas corren en la **misma reevaluación** de la operación y son independientes entre sí: ninguna llama a las otras.

2. Cuándo corre

MOMENTO	QUÉ LO DISPARA	QUÉ PRODUCE
Pre-evaluación	el ejecutivo la pide desde el detalle, con la oferta todavía abierta	adelanta el veredicto para que no comprometa un plazo a ciegas
Re-evaluación	el ejecutivo aprieta «Re-evaluar operación»	veredicto vigente + versión nueva (evidencia)
Al cerrar la oferta	el gate del curse	decide si la operación puede publicarse
Tras la firma del cliente	automático	decide si queda en <i>Otorgamiento</i> / <i>Verificación</i> o pasa a <i>Pendiente Integración</i>

La **reevaluación es explícita**. Agregar o quitar facturas **no** dispara cálculo: se actualiza sólo lo aritmético y todo lo que depende del motor queda en «Por evaluar» **sin número**, porque una cifra vieja atenuada igual se lee como cifra y alguien la va a citar.

3. El catálogo de reglas — Modelo de Riesgo v1.0

77 reglas medidas al 18-09-2026: **48 C** (cliente) · **23 D** (deudor) · **6 O** (operación). 75 vienen de la política v1.0; **005 y 006 no** — son del proceso (§6 y [Regresiones/Inconsistencias_Motor_Otorgamiento.md](#) §8).

- **183 tramos** en total: **133 de excepción**, 47 de aprobación y 3 de rechazo.
- **69 reglas** tienen al menos un tramo de excepción.
- Las excepciones rutean a **Riesgo 97 · Comercial 29 · Operaciones 7. Ninguna sin aprobador**.

El conteo vive en [atribuciones_otorgamiento.json](#), que **se regenera con un script** y no se edita: así fue como se desfasó dos meses del catálogo la vez anterior.

3.1 Cómo es una regla

Cada regla declara **el área que la aprueba** y una lista ordenada de **tramos**. Cada tramo tiene una condición sobre las variables de la operación y una **disposición**:

DISPOSICIÓN	SIGNIFICA
aprobado	el criterio se cumple, no hay nada que hacer
excepcion	se incumple y un apoderado puede autorizarlo ; el tramo declara el nivel
rechazado	se incumple y no se autoriza
clasificacion	informativa, no decide

Se evalúan **en orden** y gana el primero que calza: es un *risk tier*, donde a mayor desvío corresponde mayor jerarquía.

3.2 Reglas de cliente y reglas de deudor

Las **D** se evalúan **una vez por cada deudor** de la operación y se visan por deudor; las **C** y las **O**, una sola vez. El tipo lo **declara la regla** ([porDeudor](#)), no el prefijo del código: deducirlo del prefijo obliga a reenumerar cada vez que una regla cambia de sujeto, y se pierde la trazabilidad con la política.

Una regla dominada no entra al catálogo. La cartera del par cliente-deudor no tiene reglas propias: las mismas variables medidas a nivel de **cliente** (C40–C43) usan umbral **> 0**, así que si el par tiene un documento reclamado el cliente también lo tiene y esa excepción ya está levantada. Una regla que no puede ser la única causa de un visado se aprueba, se audita y no decide nada.

3.3 Rechazo firme y rechazo re-evaluable

Un rechazo **firmes** (no re-evaluable) es definitivo: la operación **se pierde**. Los KNOCKOUT son C30/C31/C32 (TGR judicial, convenios, cuotas impagas). Un rechazo **re-evaluable** no pierde la operación: el dato puede cambiar y se re-consulta.

La distinción importa porque la pérdida es **estado terminal**: exige causa específica, etapa de origen, actor y cierre de tareas en cascada.

4. Atribución: quién autoriza una excepción

El ruteo es el par (área, nivel). La regla declara el **área**; su tramo, el **nivel**. Con ese par se buscan los usuarios de esa área con **ese nivel o superior**.

Cinco niveles, **N5 es la máxima**. No existe un nivel «Comité»: el Comité de Crédito no es un nivel ni una cuenta del sistema.

Decisiones de negocio que rigen la búsqueda:

1. Aprueba **cualquier nivel igual o superior de la MISMA área**, sin tope.
2. Dos personas con el mismo rol aprueban las dos.
3. Un cargo **vacante** lo cubre la jefatura de su área.
4. **La escalada no cruza áreas**. Un Gerente General no visa una excepción de Riesgo.
5. Una regla con un **tramo de excepción y sin área NO SE EJECUTA** —el área es a quién se le pide la excepción, así que una **knock out** no la necesita: no se aprueba—. Desde el 18-09-2026 no se evalúa siquiera: el motor la devuelve como «No ejecutada · falta configuración», con la causa y dónde se arregla, y la operación se evalúa SIN ella. Antes se evaluaba igual y su excepción salía «Sin aprobador definido», o sea que la operación quedaba esperando a alguien que no existe. Un default silencioso escondería una regla mal configurada.

Cuando no hay nadie, el motor lo dice con esas palabras: **«Sin aprobador definido»**, con el requisito (**área · N**) y la **causa**, que son dos y se arreglan en mantenedores distintos — el área no existe en el tenant, o nadie la tiene en ese nivel o superior. Una lista de aprobadores vacía se lee como «todavía no lo miran», cuando en realidad la operación está pegada esperando a alguien que no existe.

4.1 El MONTO escala la atribución

Son **dos factores** y el requisito es **el mayor**, no la suma:

- el **tramo del risk tier** mide cuánto se desvía la variable de riesgo;
- el **piso por monto** mide cuánto se arriesga si ese desvío resulta cierto.

Sumando, dos factores medianos exigirían más que un factor extremo. Es **piso**, no reemplazo: el nivel del tramo nunca baja por el monto. Los tramos son ≤ 20 / ≤ 60 / ≤ 120 / > 120 MM.

Cada columna satura en el tope de su área — Comercial hasta N3, Riesgo hasta N5 — porque pedirle a un área un nivel que no tiene no exige más: deja «Sin aprobador definido», que sería un bug de esta tabla disfrazado de control.

4.2 La atribución sigue al ROL, no a la persona

El nivel y el área los implica el **cargo** (`ROL_ATTRIB`), no el código de usuario. Cablearlos por código de usuario sólo funciona mientras haya exactamente una persona por rol: en cuanto se le cambia el cargo a alguien, su atribución no cambia con él.

Y hay dos nociones que **no** son la misma:

- **lo que su cargo implica** — lo que se configura y lo que se delega;
- **lo que puede aprobar HOY** — que sale del **padrón**, único sitio donde se aplican los reemplazos.

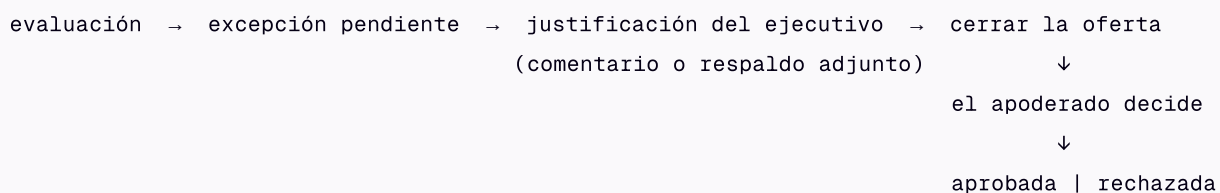
La UI tiene que mirar la segunda. Una atribución que sólo existe en el motor no existe, porque nadie llega a ejercerla: el primer intento de vacaciones tenía el motor correcto y el badge de la pantalla descartaba al reemplazante antes de preguntarle al padrón.

4.3 Vacaciones y reemplazos

Un reemplazo **agrega** un aprobador; no cambia uno por otro. Estar de vacaciones no es estar desconectado. El flag de ausencia total lo desactiva caso a caso, para la licencia o la salida, donde dejar la atribución viva es justo el riesgo — y se quita **después** de pasársela a quien cubre, o el reemplazante heredaría un cargo ya vaciado.

El fold toma el **mayor nivel por área** —cubrir a un N1 nunca baja a un N2— y marca las dos puntas, porque la bitácora tiene que poder decir **por qué** esta persona pudo aprobar esto.

5. El visado: el ciclo de una excepción



1. El motor produce las excepciones con su `(área, nivel)`.
2. El **ejecutivo justifica** cada una —comentario, respaldo, o marcar que no tiene comentarios— **antes de cerrar la oferta**: mientras quede una sin justificar, el cierre está bloqueado. La justificación no es un paso paralelo al visado sino su requisito de entrada, y va antes porque el apoderado no puede decidir sobre algo que no le llegó justificado.
3. El **apoderado** con atribución la aprueba o la rechaza.
4. El agregado de la operación es: **rechazada** si hay un rechazo firme o una excepción rechazada · **sujeta** si queda alguna pendiente · **aprobada** si todas están resueltas.

Una tarea de aprobación es para quien tenga la ATRIBUCIÓN, no para la foto de nombres del día en que se creó: guarda el par `(área, nivel)` y la lista se resuelve cada vez que se mira. El apoderado que llega la ve y el que se fue deja de verla, sin migrar nada.

El proceso operativo completo del visado —quién hace qué, en qué pantalla, qué queda registrado en cada paso y qué efecto tiene la decisión— está en `spec-gestion-excepciones.md`.

6. 005 · Contrato firmado por cliente de la operación — la regla que no viene de la política

Existe **siempre**, en las dos vías de publicación; lo que cambia es **quién crea la evidencia**:

- **Electrónica (email)**. La autorización del cliente en el portal es la evidencia; el criterio queda **aprobado** sin que nadie lo vise.

- **Física (contrato adjunto).** El papel se firmó fuera del sistema: queda como **excepción de Operaciones N3**; el ejecutivo adjunta el comprobante y Operaciones lo visa.

Que existiera sólo en la vía física dejaría a la operación electrónica cursando sin ninguna constancia mientras el cliente no firma.

La evidencia es una HUELLA del paquete, no una bandera. No dice «el cliente firmó» sino «el cliente firmó ESTO». La huella es una cadena canónica con **N° de operación · RUT del cliente · n° de deudores · n° de facturas · monto total · monto POR DEUDOR ordenado** — lo último cierra la sustitución, porque con sólo conteos y total, cambiar una factura por otra del mismo monto en otro deudor dejaría la huella idéntica. Se guarda **la cadena y su SHA-256**: el hash para comparar, la cadena para auditar — quien revise un giro en seis meses necesita ver **qué** se firmó, no un hexadecimal que sólo dice que no calza.

No entran las condiciones comerciales. El cliente firma un paquete y un monto de documentos; el precio se mueve dentro de la atribución y tiene su propio control.

Con eso el criterio se abre y se cierra solo: cambió el paquete, la huella no calza, vuelve a ser excepción. Una bandera se revoca cuando alguien aprieta «Reabrir», o sea confía en que **toda** modificación pase por esa puerta; la huella no confía en nada.

7. Qué libera el giro

Firmar es del **cliente**; girar es de la casa, y sólo después de que sus controles pasen:

ESTADO	MIENTRAS	QUIÉN ACTÚA
Otorgamiento / Verificación	falte excepcionar, llamar, o falte la evidencia del contrato	riesgo · verificación · el ejecutivo
Pendiente Integración	resuelto todo: sale del tubo y aparece en Operaciones	Operaciones
Pendiente de Giro	Operaciones N3 aprobó la integración al core	Tesorería

En el paso al core se aplica el gate de la huella: se compara la de lo que se va a inyectar contra la de la evidencia. Es el **último punto en que la comparación sirve**; después el dinero ya salió.

Parte II · El motor como servicio

8. Por qué hay que sacarlo del navegador

No es una mejora de arquitectura: es la única forma de que el motor **decida** algo.

Hoy todo el modelo de riesgo vive en el bundle que se descarga el usuario. Con la consola abierta se puede reescribir el catálogo, cambiar el nivel que exige una regla, aprobar una excepción sin atribución o mover la operación de etapa. Nada del otro lado lo impide.

Por eso el trabajo de los últimos días no fue «refactorizar»: fue conseguir que **todo lo que el motor decide entre por parámetro y salga puro**, que es la precondition para mudarlo. Un motor que lee estado global del navegador no se puede desplegar: arrastra media aplicación.

9. El contrato

9.1 evaluarOtorgamiento — la evaluación

Entrada (todo explícito; el servicio no consulta nada por su cuenta):

```
{
  "operacion": {
    "id": "OP-12345",
    "rutCliente": "76.123.456-7",
    "monto": 181900000,           // pesos, entero
    "deudores": [ { "rut": "...", "nombre": "..." } ],
    "facturas": [ { "id": "...", "rutDeudor": "...", "monto": 12345678 } ],
    "publicacion": "electronica", // | "fisica"
    "evidenciaContrato": { "huella": "op=...|rut=...|nd=...|nf=...|monto=...|deudores=[...]", "sha256": "..." }
  },
  "variables": { /* fila CLIENTE y filas DEUDOR del activo A16 */ },
  "catalogo": { /* las 76 reglas, versionadas */ },
  "politica": { /* umbrales del tenant: notaMinCompra, tramos de monto, ... */ },
  "padron": { "areas": [...], "usuarios": [...], "vigenteAl": "2026-09-14" }
}
```

Salida:

```
{
  "items": [ { "regla": 101, "codigo": "C01", "stKey": "101", "deudor": null,
    "disposicion": "excepcion", "area": "operaciones",
    "nivelTramo": 1, "nivel": 3,           // nivel tras aplicar el piso por monto
    "rolRequerido": "Jefe de Operaciones",
    "aprobadores": ["..."],              // vacío ⇒ SIN_APROBADOR, con su causa
    "hallazgo": "...", "reevaluable": true } ],
  "agregado": "sujeta",                  // aprobada | sujeta | rechazada
  "version": { "id": "...", "hash": "...", "emitida": "2026-09-14T12:00:00Z" }
}
```

Dos cosas del contrato que no son cosméticas:

- **nivelTramo** y **nivel** **viajan los dos**. El primero es lo que exige el desvío; el segundo, lo que exige después del piso por monto. Sin los dos, un apoderado no puede explicar por qué esta excepción le llegó a él y una igual del mes pasado no.
- **aprobadores** **vacío es un resultado, no un error**. Trae la causa —el área no existe en el tenant, o nadie la tiene en ese nivel— porque se arreglan en mantenedores distintos.

9.2 resolverExcepcion — el visado

Entrada: { operacionId, stKey, decision: "aprobada" | "rechazada", justificacion, adjuntos[], actor }.

Lo que el resolver tiene que hacer, aunque el cliente ya lo haya hecho:

1. **Recalcular la atribución desde el ROL DEL TOKEN**, no desde el payload. La UI que oculta el botón no es el control. La pantalla puede venir de una sesión vieja, de un rol que cambió o de una atribución revocada — y la

decisión de un apoderado es **evidencia regulatoria**.

2. **Re-evaluar la regla.** El `stKey` que llega puede no corresponder a una excepción vigente.
3. **Exigir la justificación del ejecutivo** antes de aceptar la decisión.
4. **Estampar el reemplazo** si el actor está cubriendo a otro: el campo es lo que se consulta y la glosa lo que se lee.

9.3 `avanzarEtapa` — la máquina de estados

La transición se valida **en el servidor contra la máquina de estados**; el `stage` no se acepta como dato del cliente. Con excepciones o rechazos re-evaluables sin resolver, la operación **no** pasa a cesión. En el paso al core, además, la huella tiene que calzar.

10. Los invariantes, que son el contrato mínimo

Doce códigos escritos en un solo lugar, cada uno con su **autoridad** (`servidor` = la decisión es del backend y lo del cliente es sólo anticipación; `cliente` = conveniencia de UX). El resolver los implementa **1:1 y devuelve el mismo código de error**. Los cuatro de este proceso:

CÓDIGO	REGLA	QUÉ TIENE QUE HACER EL SERVIDOR
OTG-01	sólo aprueba quien tiene atribución	recalcular la atribución desde el rol del token
OTG-02	no avanza a Cesión con excepciones pendientes	validar la transición contra la máquina de estados
GIR-02	el paquete girado es el que se autorizó	comparar las dos huellas antes de inyectar
ATR-01	descuento dentro de la atribución	recalcular la banda y el % desde la operación

La validación que hoy corre en el cliente **anticipa el rechazo, no lo impone**, y un evaluador que revienta nunca bloquea la operación: es ayuda de UX, no control.

11. Estado del aislamiento — medido, no declarado

`auditar_aislamiento.mjs` recorre las funciones que **deciden** y reporta qué estado global lee cada una, **propagando por el grafo de llamadas**. Al 14-09-2026: **36 de 85 funciones ya son extraíbles**.

La medición transitiva es la que importa. El cuerpo de una función no basta: `evaluarOtorgItems` no menciona ningún global y llama a `varsClienteActual`, que lee las versiones. El script medía sólo el cuerpo y **decía que sí**; con el grafo propagado la cifra se derrumbó, y el informe marca con  de qué *callee* hereda cada lectura, que es dónde hay que ir a arreglarlo.

Las clases con que se etiqueta cada lectura dicen qué hacer con ella:

CLASE	QUÉ ES	QUÉ HACER
<code>SESSION</code> · <code>AZAR</code>	quién está mirando · un sorteo	nunca deben decidir
<code>COMMIT</code>	evidencia con actor y hora (visado, llamadas, versiones)	inyectar : es del servidor
<code>TENANT</code>	usuarios, roles, áreas, configuración	inyectar : es del tenant, no del modelo
<code>RELOJ</code>	la hora	la pone el servidor
<code>BATCH</code> · <code>MEMO</code>	tablas precalculadas	en producción es un <code>SELECT</code>
<code>CONFIG</code>	el catálogo de reglas	es la entrada legítima del motor

Puras de verdad, cuerpo y cadena, entre las del otorgamiento: `evalReglaCli`, `deudorBlock`, `nivelExigido`, `pisoPorMonto`, `esReglaDeudor` — es decir **el núcleo evaluador**. Lo que arrastra es el borde: quién aprueba (tenant), qué se aprobó (commit) y con qué variables (batch).

Un analizador estático no puede distinguir «lee el global» de «cae al global sólo si no le pasan el estado». Por eso varias funciones figuran arrastrando estado que en realidad reciben por parámetro: el default existe como comodidad de los *call sites*. Eso **se prueba ejecutando**, inyectando un estado que **contradiga** al del navegador y comprobando cuál manda. Es lo que hacen los casos 44, 56–59 y 60–63 de la suite, y es la única evidencia que vale.

12. Lo que falta para desplegarlo

#	FALTA	POR QUÉ BLOQUEA
1	Persistencia real de los repositorios	hoy son objetos en memoria del bundle: sin <code>tenant_id</code> , sin concurrencia, sin auditoría transaccional, y se pierden al recargar. Varios son evidencia regulatoria
2	Idempotencia	hoy sólo se cuentan los duplicados; falta una clave de idempotencia que el cliente todavía no emite
3	Versionado del catálogo	el veredicto tiene que poder reproducirse con el catálogo vigente al momento de decidir , no con el de hoy
4	Concurrencia en el visado	dos apoderados decidiendo a la vez necesitan un 409 con semántica definida
5	Sacar <code>nowStamp()</code> de la evaluación	la hora la pone el servidor
6	Las variables del modelo como query , no como tabla en memoria	hoy <code>MEMO</code> / <code>BATCH</code> ; en producción es un <code>SELECT</code> del batch diario con su <code>FECHA_CORTE</code>

Ninguna es del modelo de riesgo: **son de la plataforma**. Esa es la buena noticia de la medición — lo que falta no obliga a reescribir reglas.

13. Parámetros que la política declara sin definir

No son hallazgos ni deuda técnica: son huecos de la **política**, y se cierran con Riesgo, no con código. Entre ellos: los tramos de C07, los umbrales de C08 y C37, y la semántica de O03. Están listados uno por uno en `Regresiones/Inconsistencias_Motor_Otorgamiento.md` §5.

Documentos relacionados

DOCUMENTO	QUÉ CUBRE
Inconsistencias_Motor_Otorgamiento.md	la auditoría de la política contra la implementación, y §8, 005
Revision_Definiciones_2026-09-11.md	qué debe recoger la próxima versión del PDF de política
spec-gestion-excepciones.md	el proceso operativo de una excepción: justificación, solicitud, bandeja, decisión, efectos y evidencia
spec-verificacion-facturas.md	la segunda rutina de la misma reevaluación
spec-asignacion-lineas.md	la tercera: cuánto cupo hay
spec-pricing-simulacion.md · spec-modelo-giro.md	lo que viene después del otorgamiento
Integraciones/spec_s3_otorgamiento.md	el layout A16: las variables que alimentan el motor
Levantamiento_Activos_Informacion.md	el inventario A1–A23 y, en §5, qué activo es maestro de cada campo

Anexo · Control de versiones

Mayor = cambia lo que el sistema decide o el contrato con el servidor · **menor** = entra una sección, un campo o un criterio · **parche** = redacción, una cifra o una referencia.

VERSIÓN	FECHA	QUÉ CAMBIÓ
1.2.1	21-09-2026	Rutas de los documentos citados, tras agrupar la documentación por carpetas.
1.2.0	18-09-2026	Una regla con un criterio sin área —o sin nadie que pueda firmar su excepción— no se ejecuta ni se verifica, y la salida dice la causa y en qué mantenedor se arregla.
1.1.0	16-09-2026	El diagrama del visado corrige el orden: la justificación va antes de cerrar. Las entregas diarias que alimentan el motor pasan de SFTP a S3.
1.0.0	14-09-2026	Primera versión: el proceso de otorgamiento y su contrato como servicio, en un solo documento.